

第三章复习

- 三级调度
- 周转时间、带权周转时间
- 先来先服务、短作业优先、高响应比、时间片轮转
- 多级反馈队列调度：抢占式
- 最低松弛度优先：抢占式
- 死锁概念、原因
- 死锁四个必要条件
- 解决死锁的四种方法：预防、避免、检测、解除
- 预防死锁三种方法：破坏三个必要条件
- 避免死锁：银行家算法、安全状态、安全序列
- 死锁检测：资源分配图，死锁定理

练习

- 已知系统当前有资源**ABCD**，正在运行**5**个进程**P0**、**P1**、**P2**、**P3**、**P4**。在**T0**时刻，资源分配如下表所示。系统若采用银行家算法实施死锁避免策略，试问：
 - **T0**时刻是否为安全状态？请给出分析过程，若安全请给出安全序列。
 - 若**P1**提出请求（**1, 1, 1, 0**），系统能否分配资源给它？请给出分析过程，若可以分配请给出安全序列。

	Allocation				Need				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	3	2	0	0	1	2	1	2	2	2
P1	1	2	0	0	1	6	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	3	3	2	0	2	5	2				
P4	0	1	1	4	0	5	5	6				

第四章 存储器管理

- 4.1 存储器层次结构
- 4.2 程序的装入和链接
- 4.3 连续分配方式
- 4.4 基本分页存储管理方式
- 4.5 基本分段存储管理方式
- 4.6 虚拟存储器的基本概念
- 4.7 请求分页存储管理方式
- 4.8 页面置换算法
- 4.9 请求分段存储管理方式

多级存储器结构

- 通用存储层次：CPU寄存器、主存、辅存
- 具体功能划分：
 - 寄存器
 - 高速寄存器（Cache）
 - 主存储器
 - 磁盘缓存（非实际存储介质）
 - 固定磁盘
 - 可移动存储介质

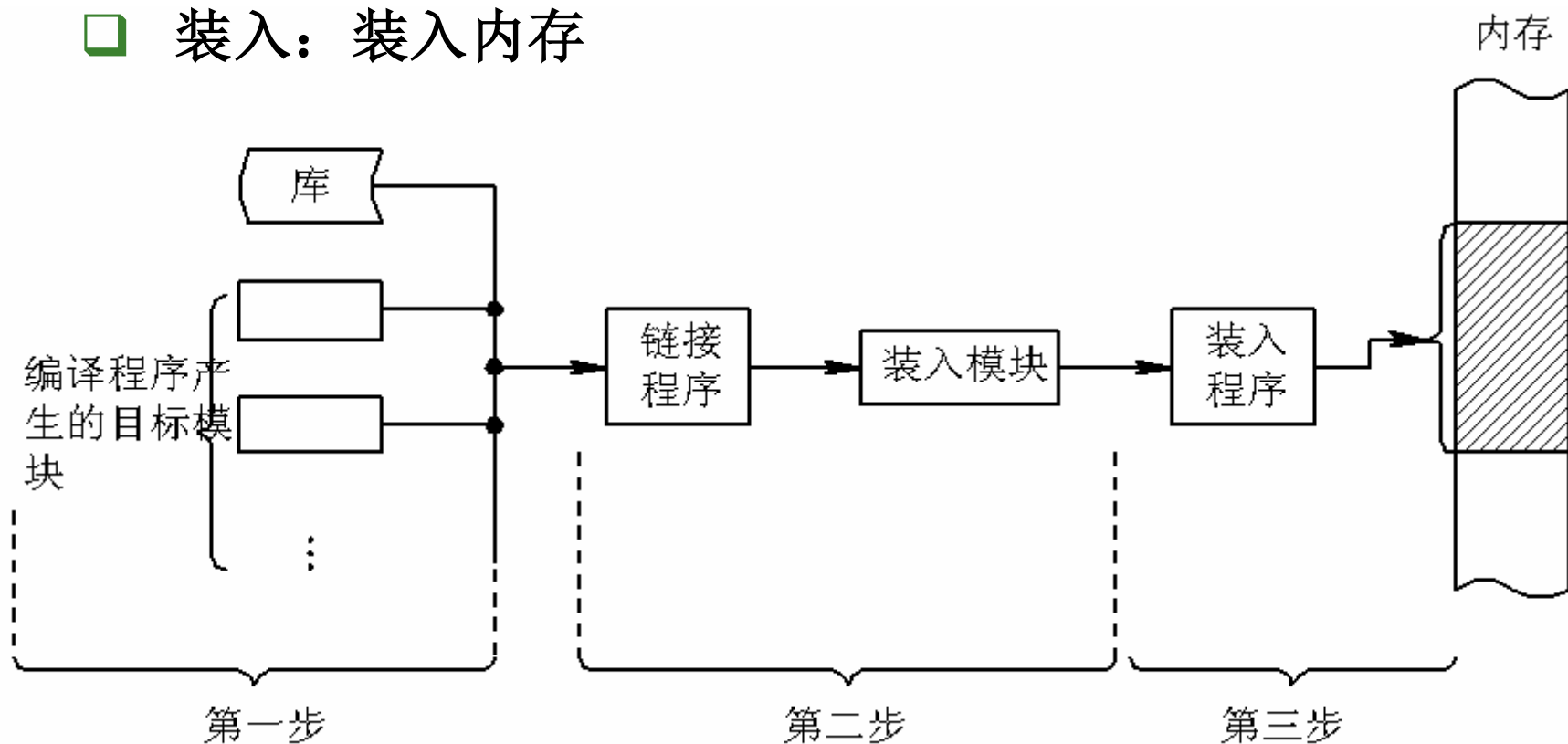
程序的装入和链接

- 存储器管理对象：内存
- 管理目标：
 - 为用户使用存储器提供方便，用户只需要在自己的逻辑内存空间编程，不需要考虑真实内存的物理位置
 - 为用户提供充分大的存储空间
 - 提高内存的利用率，将尽量多的用户调入内存运行
- 逻辑地址与物理地址的含义、关系

程序的装入和链接

■ 程序进入内存的流程步骤

- 编译：源代码→目标模块
- 链接：目标模块+库函数→装入模块
- 装入：装入内存



程序的装入和链接

■ 程序的装入

- ❑ 绝对装入：绝对映射，程序中逻辑地址与内存物理地址完全相同
- ❑ 可重定位装入：静态映射，在装入时对逻辑地址进行修改
- ❑ 动态运行时装入：逻辑地址到物理地址的映射在程序运行时才执行

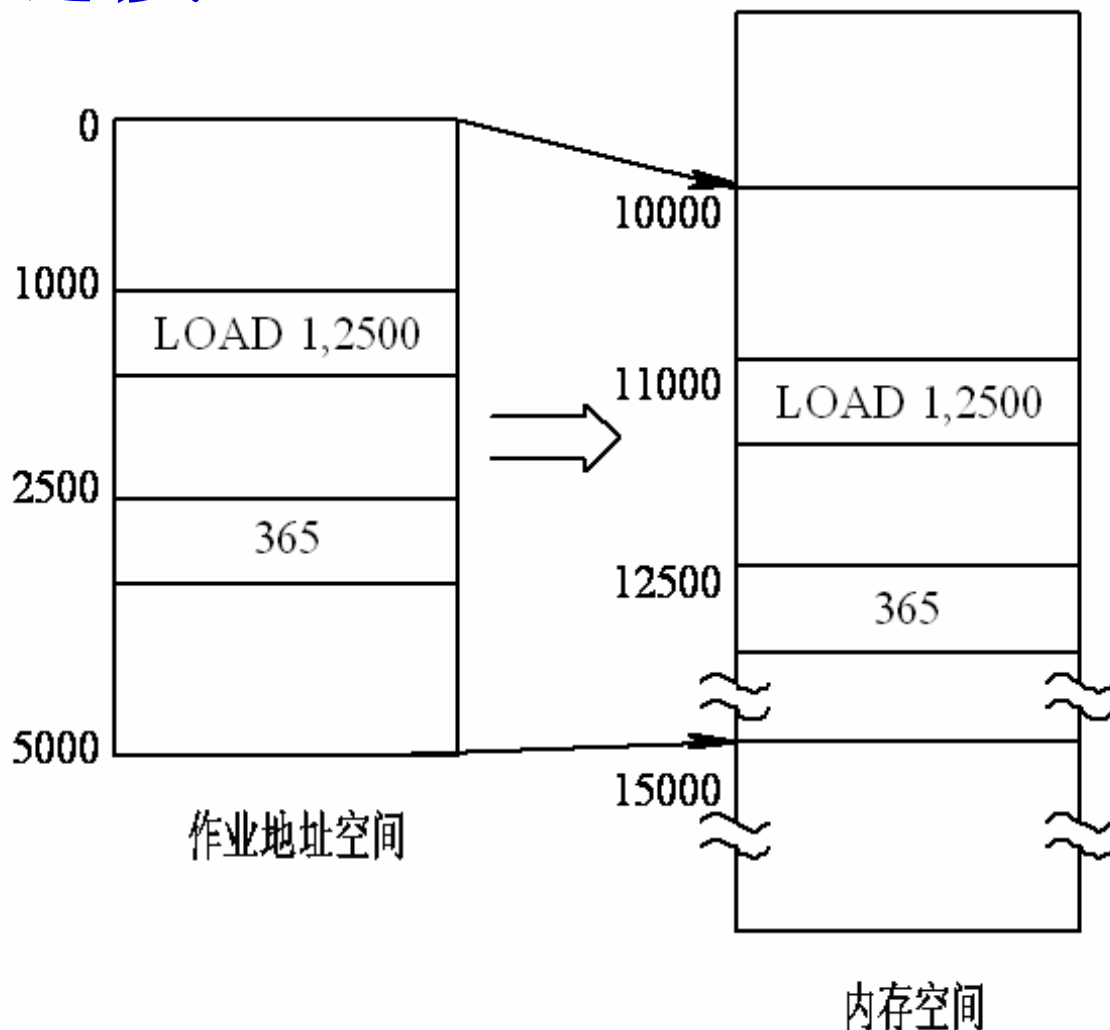
程序的装入和链接

■ 绝对装入

- 程序中所使用的绝对地址，既可在编译或汇编时给出，也可由程序员直接赋予。
- 通常是在程序中采用符号地址，然后在编译或汇编时，再将这些符号地址转换为绝对地址。
- 特点：程序的逻辑地址与实际内存地址相同

程序的装入和链接

- 可重定位装入
 - 在装入时对逻辑地址做一次性修改，以后不再改变
- 动态运行时装入
 - 地址转换到程序真正要执行时才进行
 - 装入内存后的所有地址都仍是相对地址

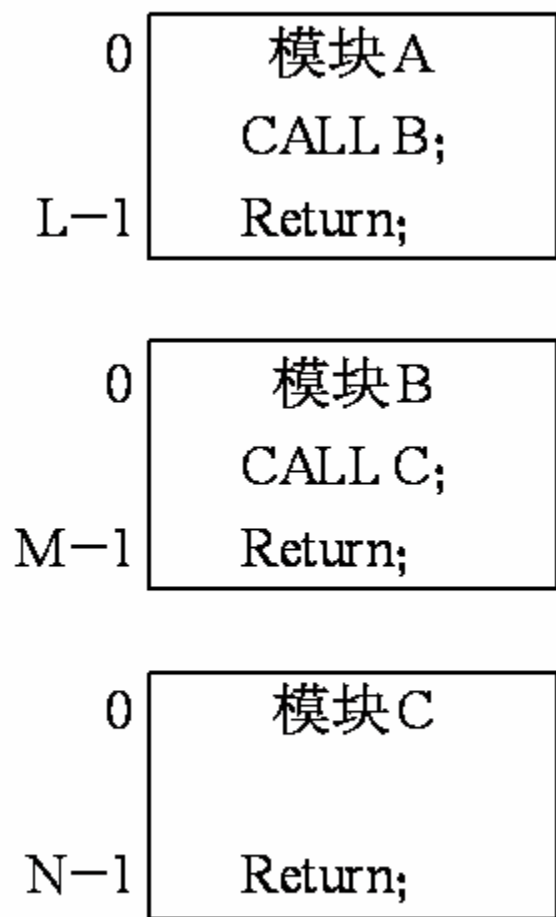


程序的装入和链接

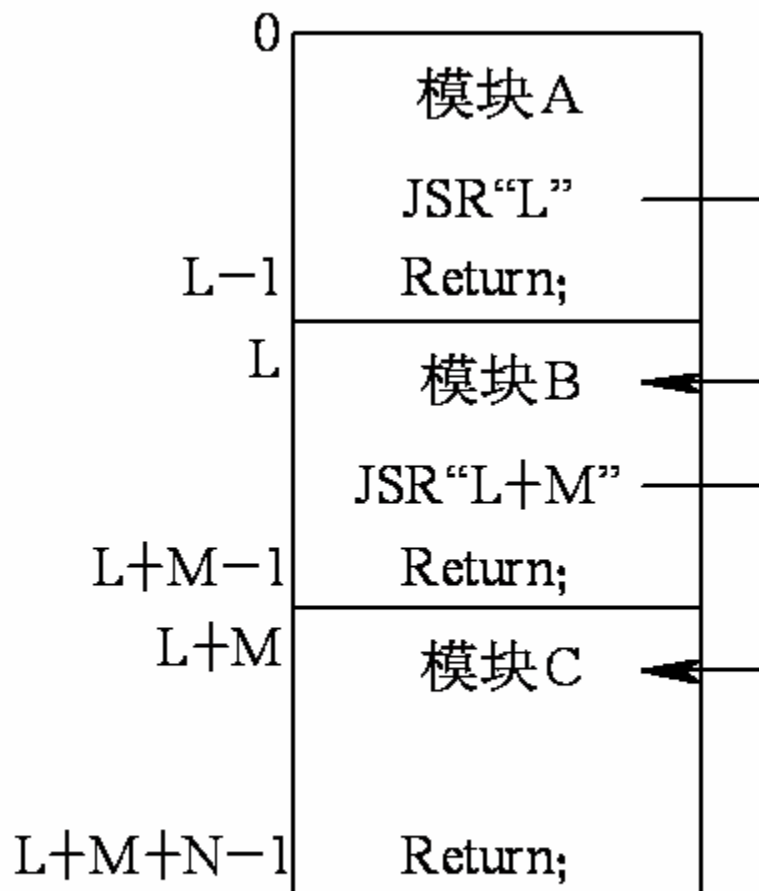
- 程序的链接
 - 静态链接方式(**Static Linking**)
 - 装入时动态链接(**Load time Dynamic Linking**)
 - 运行时动态链接(**Run-time Dynamic Linking**)

程序的装入和链接

- 静态链接：在装入内存前已经链接成一个整体



(a) 目标模块



(b) 装入模块

程序的装入和链接

■ 装入时动态链接

- 一边装入一边链接
- 便于修改和更新
- 便于实现对目标模块的共享

■ 运行时动态链接

- 执行过程中，当发现一个被调用模块尚未装入内存时，立即由**OS**去找到该模块并将之装入内存，把它链接到调用者模块
- 不用的目标模块不会被调入内存和被链接到装入模块
- 加快程序的装入过程，而且可节省大量的内存空间

连续分配方式

- 连续分配：为每个用户程序分配连续的内存空间
 - 单一连续分配
 - 固定分区连续分配
 - 动态分区分配
 - 动态重定位分区分配

连续分配方式

■ 单一连续分配

- 一个用户程序独占连续的内存用户空间
- 用于单用户，单任务的**OS**中
- 内存只分为两个区：系统区和用户区
- 一般对系统区作保护，用基址寄存器做控制

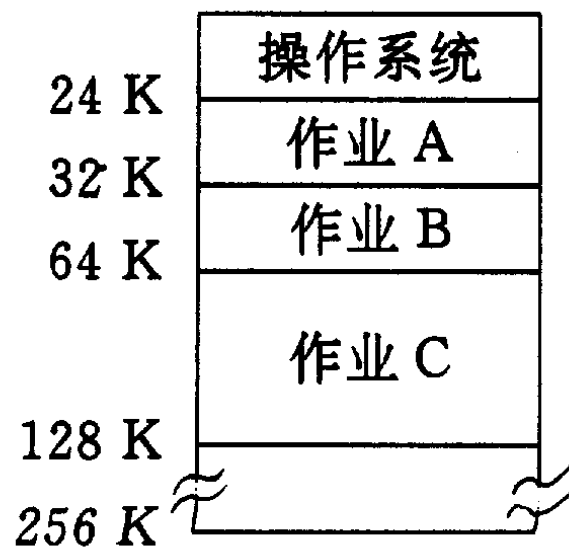
连续分配方式

■ 固定分区分配

- ❑ 将内存划分多个区域，每个区域大小固定
- ❑ 每个区域只能装入一个程序
- ❑ 分区大小相等的划分方法，一般用于多个相同任务
- ❑ 分区大小不等的划分方法，根据实际任务大小分配

分区号	大小(K)	起址(K)	状态
1	12	20	已分配
2	32	32	已分配
3	64	64	已分配
4	128	128	已分配

(a) 分区说明表



(b) 存储空间分配情况

连续分配方式

■ 固定分区分配容易造成浪费

有A、B、C、D、E五个作业，大小分别为7K，24K，20K，40K，90K。
A、B两作业先分别装入内存区1、3运行，C、D、E处于后备队列

FF算法分配情况

区号	大小	始址	状态
1	8K	16K	A占用
2	16K	24K	可用
3	32K	40K	B占用
4	64K	72K	可用
5	120K	136K	可用

区号	大小	始址	状态
1	8K	16K	A占用
2	16K	24K	可用
3	32K	40K	B占用
4	64K	72K	C占用
5	120K	136K	D占用

空闲区(2、4、5)200K只
分配使用60K (30%)

连续分配方式

■ 动态分区分配

- 无固定分区，在整块内存空间中动态分割
- 无用户程序，内存是一整块连续分区
- 根据程序大小，动态地分配连续的内存空间，分配的内存大小等于程序大小

■ 要求解决的问题

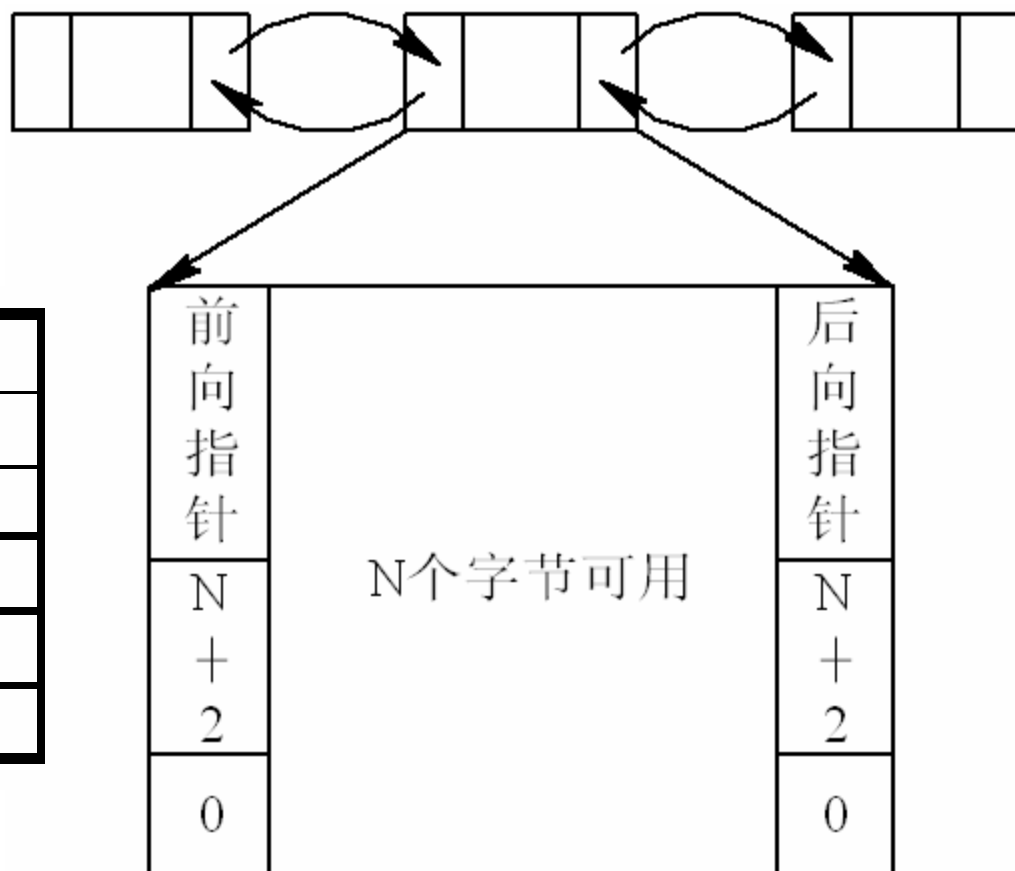
- 分区数据结构
- 分配算法
- 分配和回收操作

连续分配方式

■ 动态分区分配数据结构

- 空闲分区表
- 空闲分区链

区号	大小	始址	状态
1	64K	44K	可用
2	24K	132K	可用
3	40K	210K	可用
4	30K	270K	可用
5	...	...	...



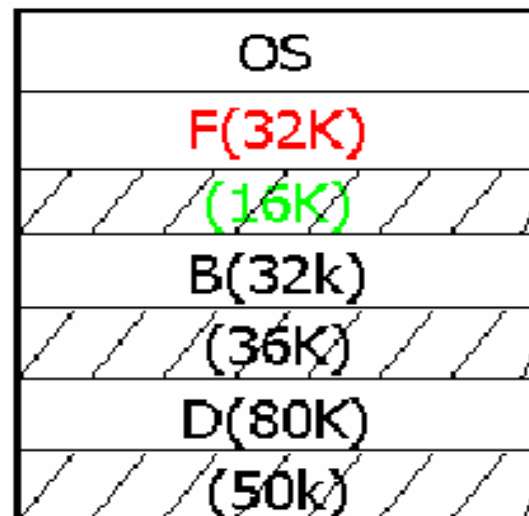
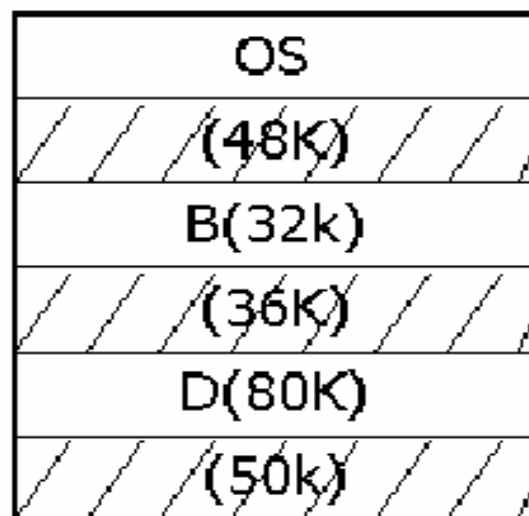
连续分配方式

■ 分配算法

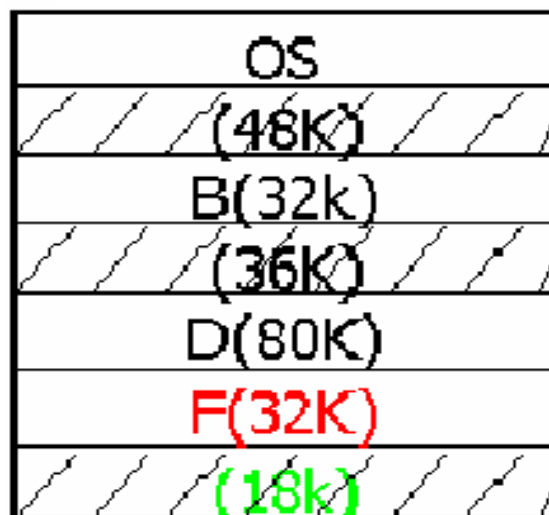
- 首次适应：按顺序选择第一个满足要求的内存区域
 - 优点：保留高地址的大部分空间
 - 缺点：低地址存在很多小片无法利用的空间，查找时间长
- 循环首次适应：在上一次找到空闲分区的下一个分区开始寻找，找到第一个满足要求的内存区
 - 优点：空闲分区在内存中分布均匀，查找时间短
 - 缺点：缺乏大的内存空间
- 最佳适应：在所有空闲分区中查找与程序大小最相近的空闲分区
 - 为了加快速度，可以把空闲分区从小到大排列
 - 优点：提高内存使用率，保留大空闲去
 - 缺点：仍然会存在小片无法利用的空闲分区

连续分配方式

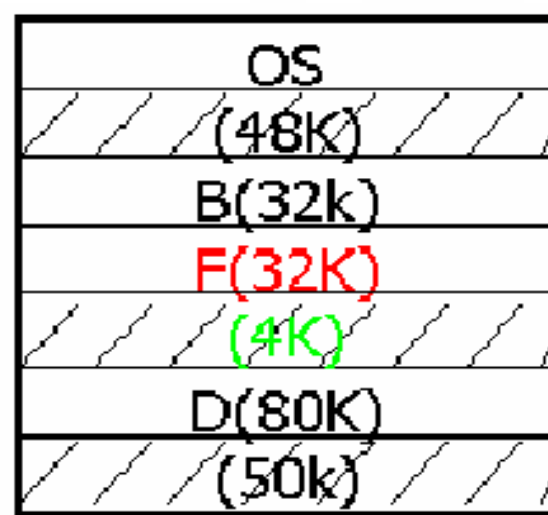
- 假设有新程序F装入，大小为32K，当前已有程序B和D，最近一次空间分配是D



FF



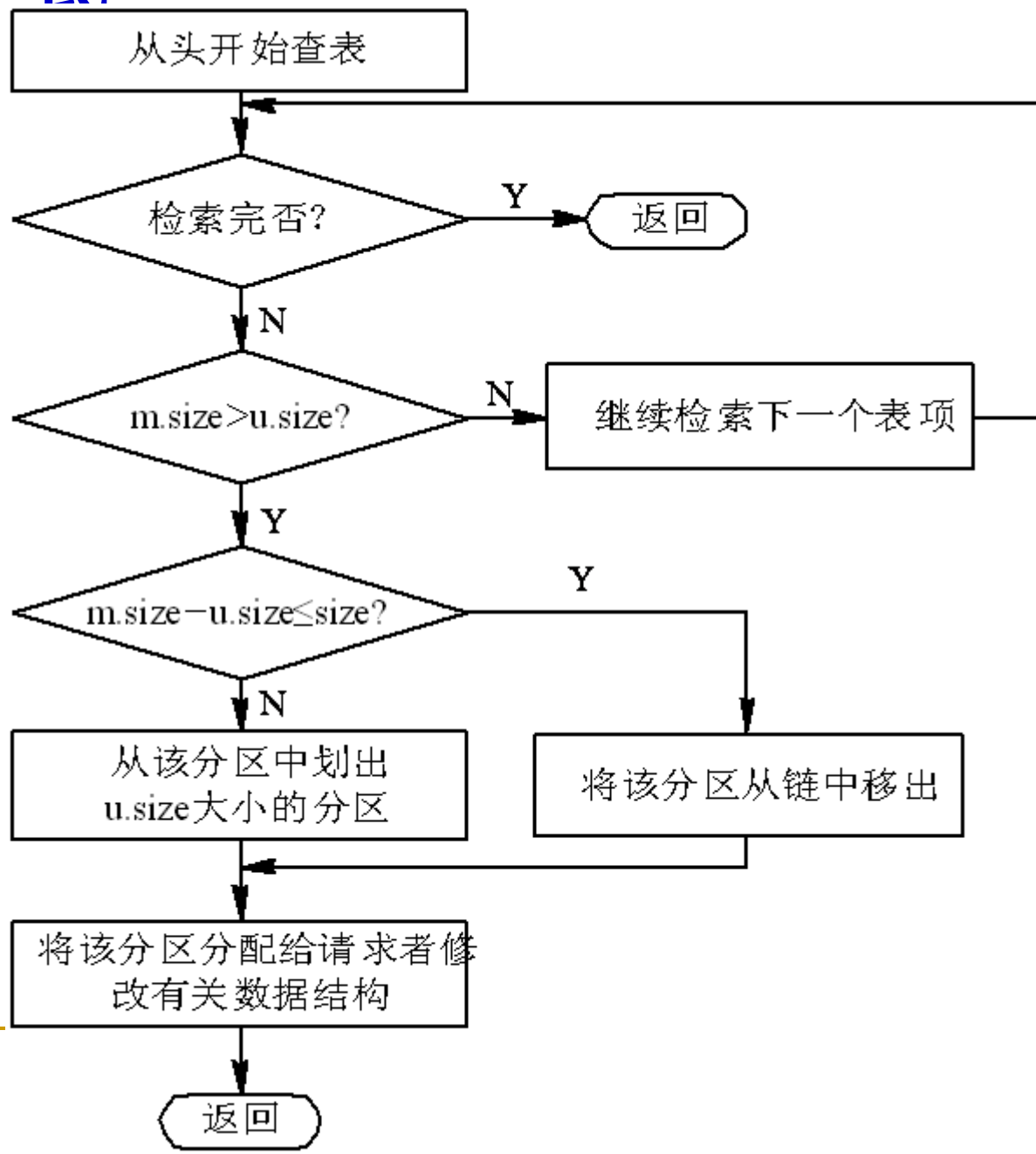
循环



BF

连续分配方式

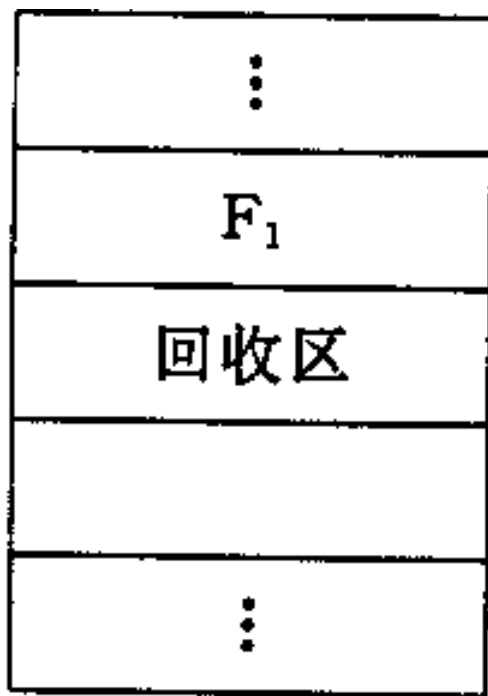
分配内存



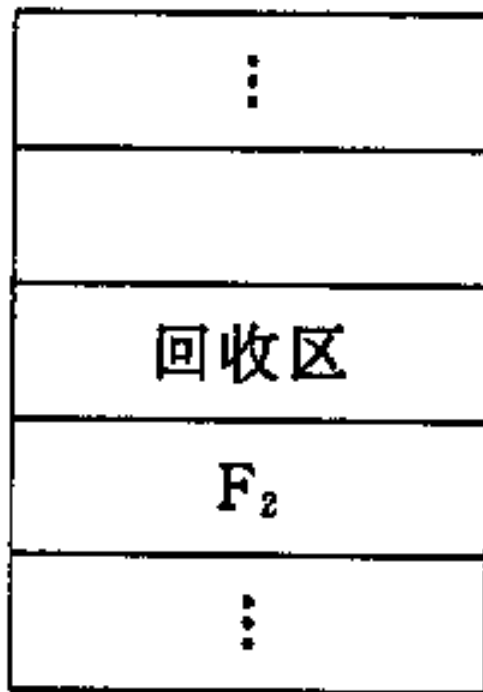
连续分配方式

■ 回收内存

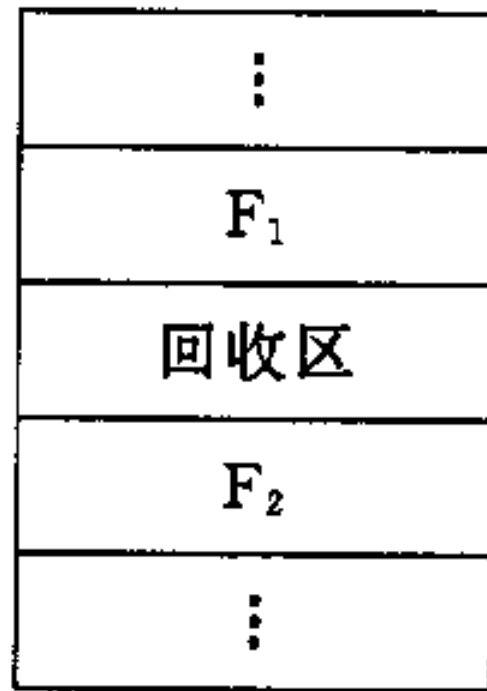
□ 考虑三种情况



(a)



(b)



(c)

连续分配方式

- 伙伴系统：分割与回收均按2的K次幂进行分类操作

- 可重定位分区分配：对内存中正在使用的分区进行搬迁，使多个小的空闲分区（碎片）合并为一个大的空闲分区



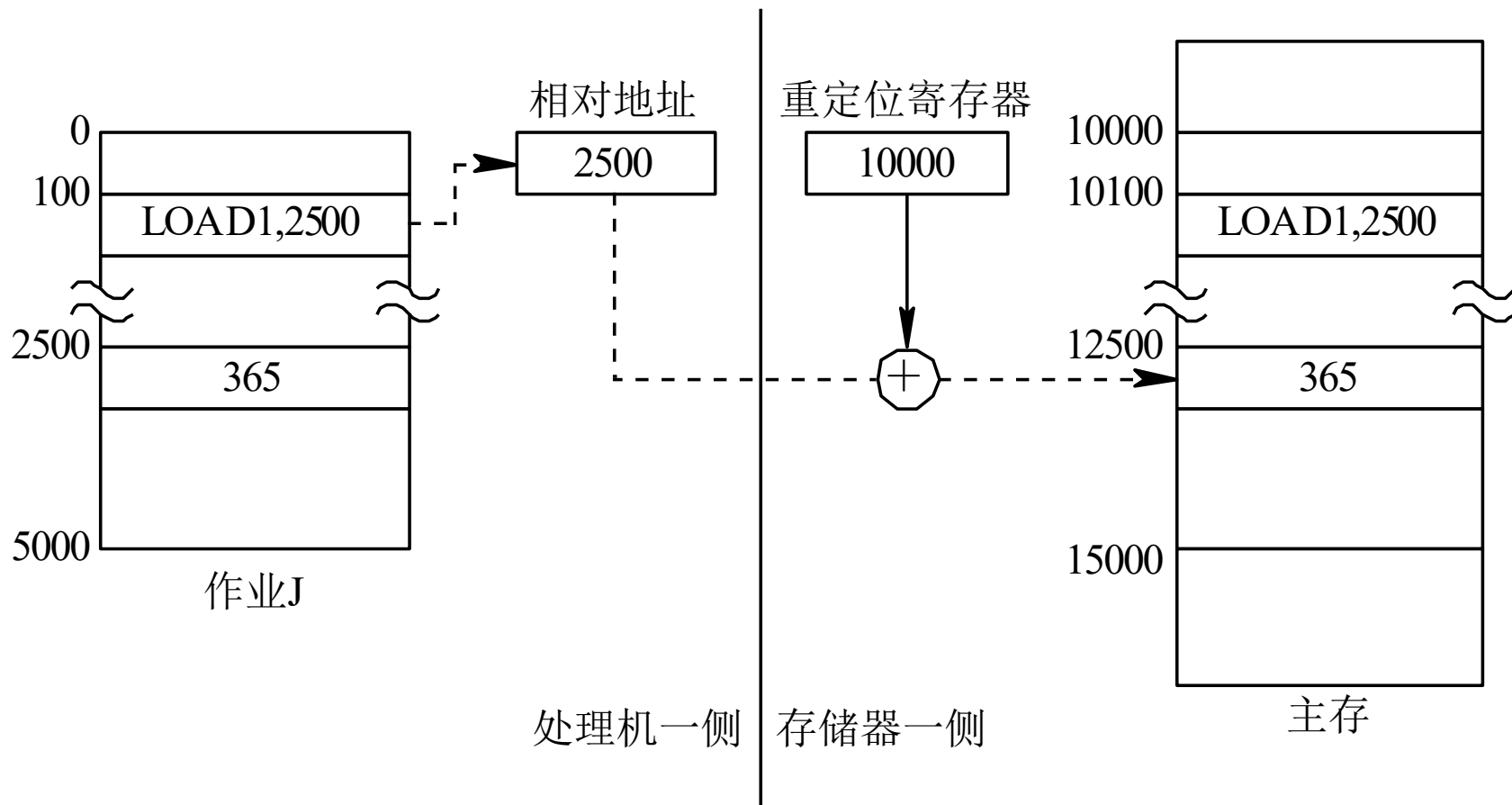
(a) 紧凑前



(b) 紧凑后

连续分配方式

■ 动态重定位的实现



连续分配方式

■ 动态重定位分区分配算法

